

Interro – Mocks et Tests Unitaires en Python

Contexte de l'énoncé :

Vous travaillez sur un projet de ruche intelligente. L'un des modules permet de surveiller la température interne de la ruche à l'aide d'un capteur thermique. Si la température dépasse un certain seuil, un système de ventilation se déclenche automatiquement pour refroidir l'intérieur de la ruche.

Voici le comportement attendu :

- Si la température mesurée est supérieure à 35°C, le ventilateur doit être activé.
- Sinon, il reste éteint.

Le code initial ressemble à ceci :

```
def lire_temperature():
    """Lecture de la température depuis un capteur (en °C)"""
    pass # à implémenter avec le vrai capteur

def controle_ventilation():
    temperature = lire_temperature()
    if temperature > 35:
        return "VENTILATEUR ON"
    return "VENTILATEUR OFF"
```

Questions

Question 1.

Vous devez structurer le système de surveillance de votre ruche connectée. Pour cela :

- Commencez par créer un modèle de base que l'on pourrait réutiliser pour différents éléments physiques (capteurs, actionneurs...). Ce modèle contient au moins deux caractéristiques communes que ces objets partagent (exemples : nom, port de connexion...).
- Ensuite, créez un capteur de température qui se base sur ce modèle commun.
- Enfin, comme certains objets du système doivent pouvoir envoyer ou recevoir des informations, vous devez proposer une structure claire qui oblige ces objets à définir des fonctions de lecture et d'action. Cette structure pourra être utilisée autant pour le capteur que pour un éventuel système de ventilation. Celle-ci ne contient que des comportements.

Question 2. Complétez un test unitaire utilisant unittest et un mock pour simuler un cas où la température est de 37°C. Le test devra vérifier que la ventilation est bien activée.

```
import unittest
from unittest.mock import patch

class TestControleVentilation(unittest.TestCase):
    pass

# Ne pas oublier ceci pour exécuter les tests
if __name__ == "__main__":
    unittest.main()
```

- (a) Complétez les informations manquantes.
- (b) Expliquez **en une phrase claire** pourquoi on utilise un mock ici.

Question 3. Écrivez un deuxième test unitaire pour simuler une température de **30°C** et vérifier que la ventilation ne s'active pas.

Question 4. Expliquez la différence entre utiliser `@patch(...)` en décorateur et utiliser `with patch(...)` dans un test. Donnez un exemple de situation où l'un est préférable à l'autre.

Bonus

Et si maintenant on voulait simuler une **erreur du capteur** (ex. `lire_temperature()` lève une exception `ValueError("Capteur HS")`), que faudrait-il ajouter dans le test pour s'assurer que l'exception est bien levée ?